MSCS-532

Algorithms and Data Structures

Spring 2026

Name: Victor Hugo Abad Navarrete

Assignment 6: Medians and Order Statistics & Elementary Data Structures

1. Theoretical Analysis

1.1 Selection algorithms

Deterministic Algorithm (Medians of Medians)

The median of medians selection algorithm provides a predictable upper bound in running time complexity by guaranteeing to choose the median value of a subarray as the partitioning pivot, hence avoiding the worst-case scenarios that would produce a $O(n^2)$ running time complexity. The total running time T(n) is the sum of the running time of all the steps of the algorithm. The initial loop discarding elements until the subarray is divisible by 5 runs in time O(n) since the comparison is performed for all the elements of the subarray minus 1. The sorting (in groups of 5) step has a complexity of O(n) since sorting 5 elements can be considered as a constant factor, and this step runs n/5 times. The recursive select step to find the pivot runs over the middle region, which contains all the medians (T(n/5)) while the recursive select step runs at most 7n/10 times (Cormen, Leiserson, & Rivest, 2022). Hence, the total cost is:

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n) + O(1)$$

By using T(n) = O(n) using the substitution method, the inequality T(n) <= cn must hold:

$$T(n) \leq \frac{cn}{5} + \frac{7cn}{10} + O(n) + O(1)$$

$$T(n) \leq \frac{9cn}{10} + O(n) + O(1)$$

$$T(n) \leq cn$$

Randomized Selection

The randomized selection algorithm chooses a random pivot to partition the array and perform recursive randomized select based on whether the pivot was higher or lower than the required i-th element to be selected. The problem is described probabilistically as the probability that a Bernoulli trial is useful for selecting the pivot in the middle half. The selection required two trials on average, and each successful one reduces the number of elements still in play by at least 3/4. The variable A(j) is the set of elements that are still on play after j partitions. A partitioning step is helpful if the number of elements of the next partitioning is less than or equal to 3/4 of the previous step (Cormen, Leiserson, & Rivest, 2022):

$$\left|A^{(j)}\right| \leq \frac{3}{4}\left|A^{(j-1)}\right|$$

A helpful partition is given by $A^{(h_k)}$, $A^{(h_k+1)}$,…, $A^{(h_k+X_k-1)}$m where XK is the random variable $X_k = h_{k+1} - h_k$The total number of expected comparisons in the algorithm determines the running time. This is given by the sum:

$$E(X_k) \leq \sum_{k=0}^{m-1} \sum_{j=h_k}^{h_k+X_k-1} |A^{h_k}|$$

$$E(X_k) = \sum_{k=0}^{m-1} X_k |A^{h_k}|$$

$$E(X_k) \leq \sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0$$

Where n0 is the size of the original array. Since the number of expected trials to render a helpful partitioning is less or equal than 2 ($E[X_k] <= 2$)

$$E(X_k) = \sum_{k=0}^{m-1} E[X_k \left(\frac{3}{4}\right)^k n_0]$$

$$E(X_k) = n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k E[X^k]$$

Substitute $E[X_k] <= 2$:

$$E(X_k) \leq 2n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k$$

$$E(X_k) \leq 2n_0 \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k$$

$$E(X_k) \leq 8n_0$$

Hence, the total number of expected comparisons is linear with the total number of elements the array, so the expected running time is $\Theta(n)$.

1.2 Data structures

Array and Matrix

Arrays and matrices are implemented as contiguous blocks of memory, each of the same size and data type; any operation that modifies the array must maintain this property. The translation from a matrix to a one-dimensional address space is handled automatically by the compiler.

1. Insertion and deletion

For insertion and deletion, the worst-case scenario happens when the required element is the fist one. Then, each element of the original array must be shifted to maintain the same order in the array. Also, for the insertion case, it is possible that the memory in both sides of the array is already allocated, requiring moving the entire array to a new memory location with enough space for all the original elements plus the new element to be inserted. Since, in the worst case, all the elements need to be shifted/copied, the time complexity is O(n).

2. Access

For access, the address in memory is calculated from the access index as simple arithmetic. Hence, the time complexity is constant O(1).

Stack and Queue

Stacks and queues implemented from arrays use a head and a tail pointer to keep track of the first and last elements of the structures.

1. Push/Pop

Push and pop only require accessing or updating an element calculated from the head pointer of the stack, and updating the head pointer value. Both operations only have arithmetic/memory constant costs, so the upper bound is constant O(1).

2. Queue/Dequeue

Queuing and dequeuing imply accessing or updating an element pointed by the head or tail pointers, and updating the head or tail pointers. Both operations have a constant cost, so the upper bound is constant as well, O(1).

Linked list

The linked list structure consists of a series of elements that reference other elements to create a sequence that defines the structure. Each element contains a pointer to the previous element, to the next element, the value of the element, and optionally other fields.

1. Insertion/Deletion

For insertion, first a new element needs to be created in memory. Then, based on which position it needs to be placed, the neighboring elements' pointers need to be updated to include the new element, and the pointers of the new element should reference the neighboring elements. For deletion, it is only required to update pointers of the neighboring elements to the element to be deleted, and then deallocate memory for the element being deleted. For both insertion and deletion, the cost is constant, provided the insertion or deletion position has already been determined. Hence, the time complexity is O(1).

2. Traversal

To retrieve an element from the linked list, the worst-case scenario happens when the element is at the opposite end of the linked list from the end from which the list starts to be examined. Since the linked list sequence is unpredictable, all elements need to be examined until a match is found. Hence, the time complexity for traversal is bounded by O(n).

Discussion

The three types of structures described (arrays, queues/stacks, linked lists) have different characteristics that make them more suitable for specific applications than others. In particular, the speed of the operations and flexibility determine their usage for a specific application. Arrays are

suited for applications that require fast access to any element by its index, and also when the data size is bounded. In fact, some programming languages like C do not allow the modification of the size of an array once it has been allocated. Relevant applications can include general scientific and discrete element analysis, where matrix operations are frequent.

Stacks and queues only support a specific data access pattern, and they are very efficient in doing it. Inserting and retrieving elements from those data structures is very fast, and the data organization follows an order that is beneficial for certain applications. Queues allow for the implementation of FIFOs, which are the basic elements to pass information asynchronously between different domains. Concrete applications can include network transfers or system buses (e.g., AXI). These data structures should be limited to only the functions for which they are designed. Querying arbitrary elements is possible but inefficient, and it is also required to have access to the structure's start and end pointers.

Linked lists are beneficial for uses where arbitrary data insertion and deletion are performed. However, querying (traversing) the linked list could require checking all the elements of the list since there is no information about the ordering of the elements besides the head element. Also, linked lists require more memory than arrays, stacks, and queues, since each element must store references to the preceding and succeeding elements. The properties of linked lists make them suitable for applications where an arbitrary relationship between the elements is very likely, such as graph databases and hierarchical structures.

2. Implementation and comparison

The median of median and random pivot selection algorithms, together with the data structures, were implemented in the following repo:

https://github.com/victorvic-ucumberlands/MSCS532_Assignment6

Selection algorithms

Figures 1-4 show the performance and memory usage of the median-of-medians and random-pivot selection algorithms. The median-of-medians selection algorithm performs consistently worse than the random-pivot algorithm, both in time to completion and in memory utilization. For the largest arrays, the median-of-medians algorithm is about 6 times slower than the random-pivot selection algorithm. The additional overhead incurred by the median-of-medians selection algorithm significantly slows down the run. Importantly, the pivot selection step itself creates a new recursive path, while the group-wise ordering section requires accessing data with little or no locality, since the elements will be very far apart for large arrays. Therefore, while in theory the median of medians selection algorithm provides a lower upper bound than the random pivot selection algorithm, the real overhead of the extra steps makes it perform worse in all cases.

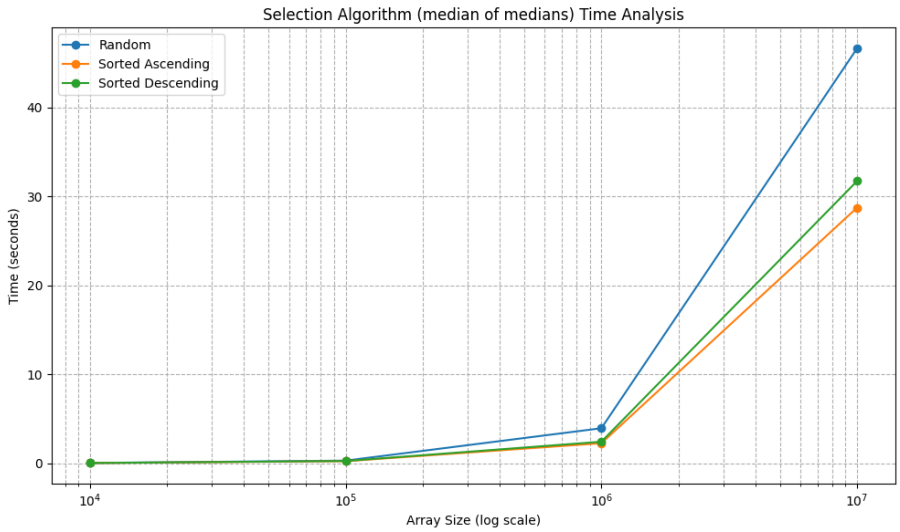

Figure 1: Performance of the median of medians selection algorithm

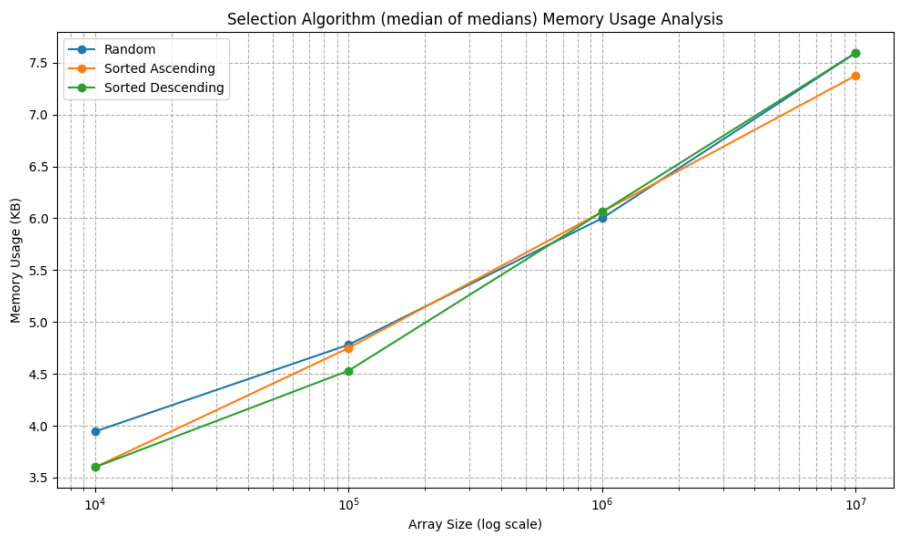

Figure 2. Memory consumption of the median of medians selection algorithm

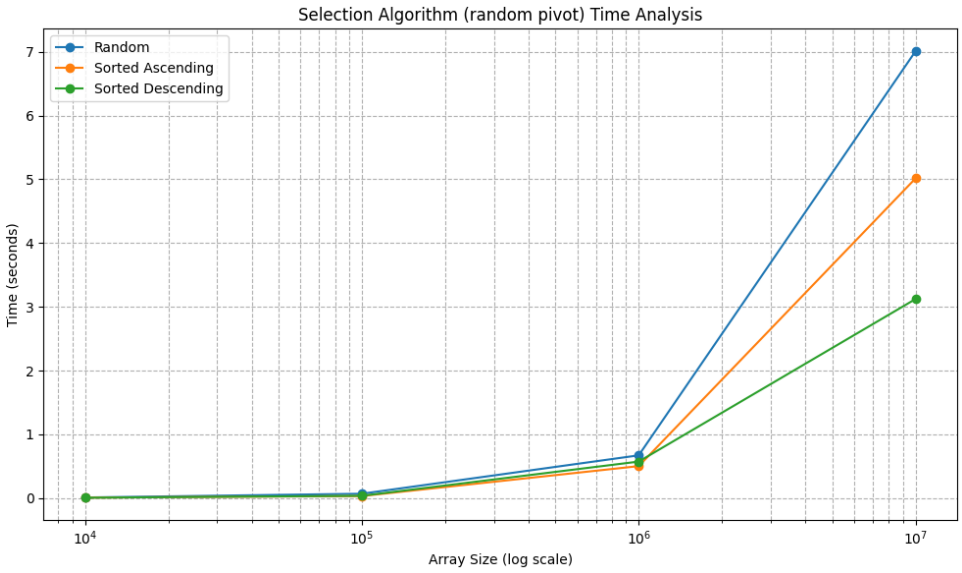

Figure 3: Performance of the random pivot selection algorithm.

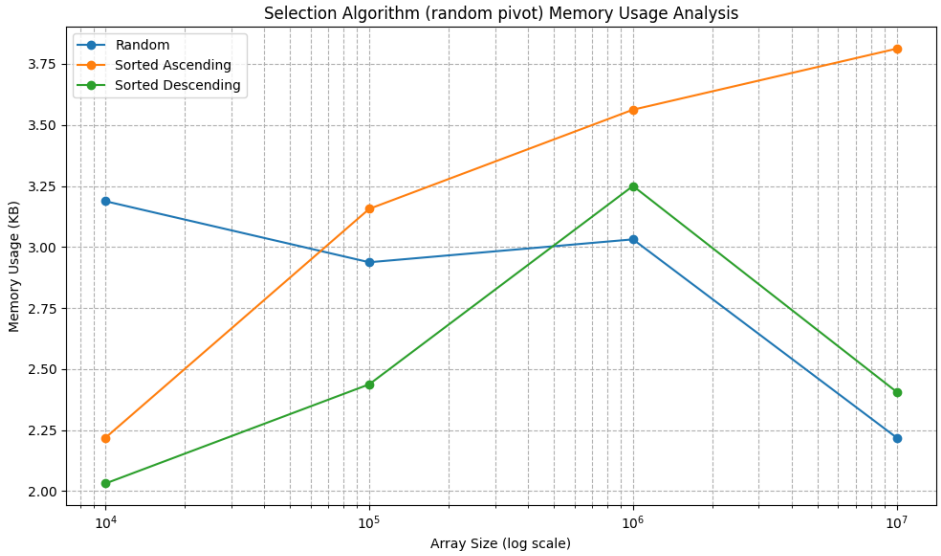

Figure 4: Memory consumption of the random pivot selection algorithm.

Data structures

Figures 5-9 show the performance of the different elementary data structures in their basic operations. Notably, in all cases, the first operation performed on the entire array (insert, push, enqueue) was slower than subsequent operations, except for linked lists. This is due to the data's temporal locality. Once the entire structure has been accessed the first time, it is cached and faster for subsequent access. Also, for the linked list, the retrieval step was very slow. This is because for every single retrieval, the linked list must be traversed to find the required element.

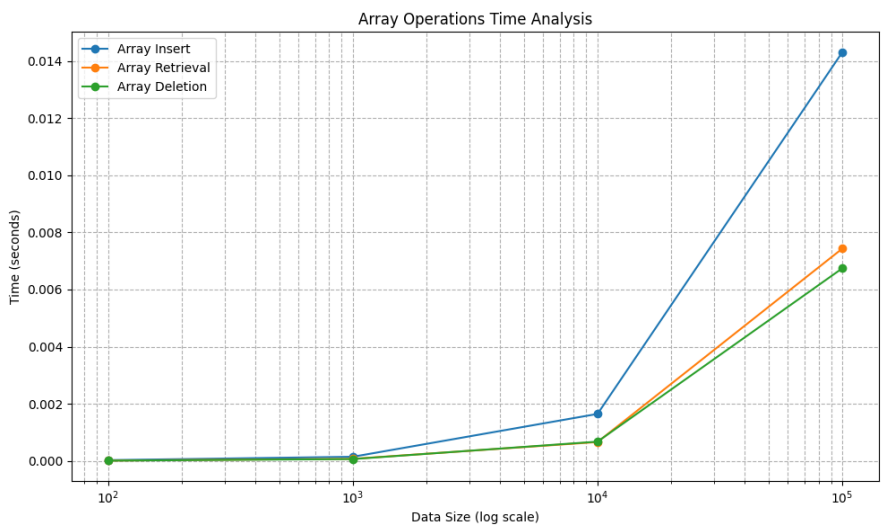

Figure 5: Performance of operations in array data structure.

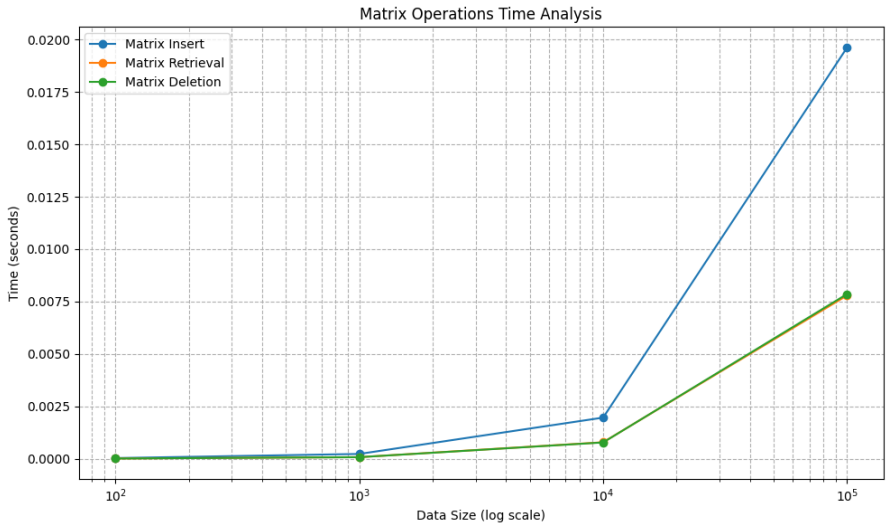

Figure 6: Performance of operations in matrix data structure.

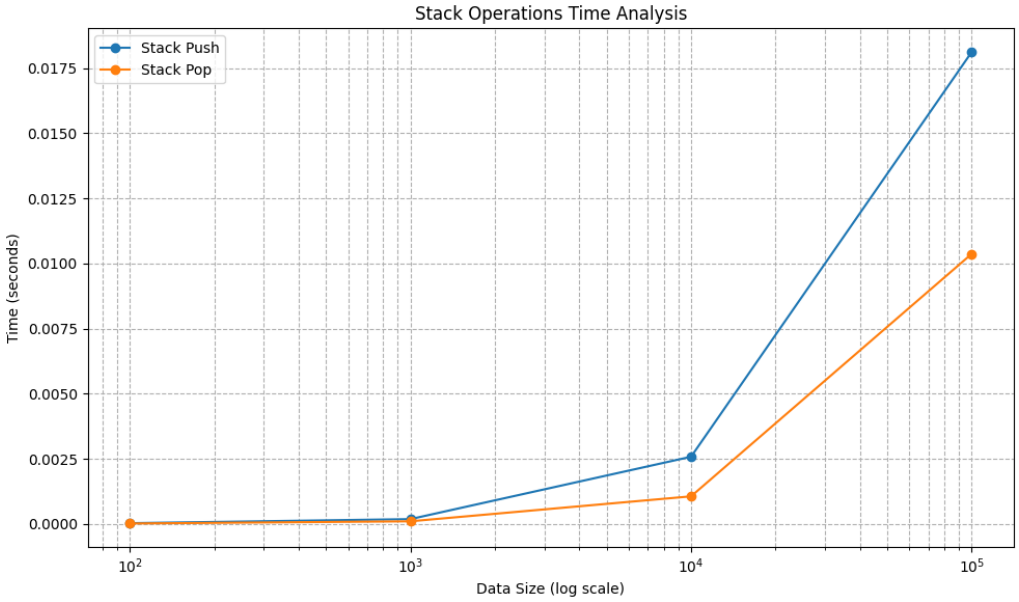

Figure 7: Performance of operations in stack data structure.

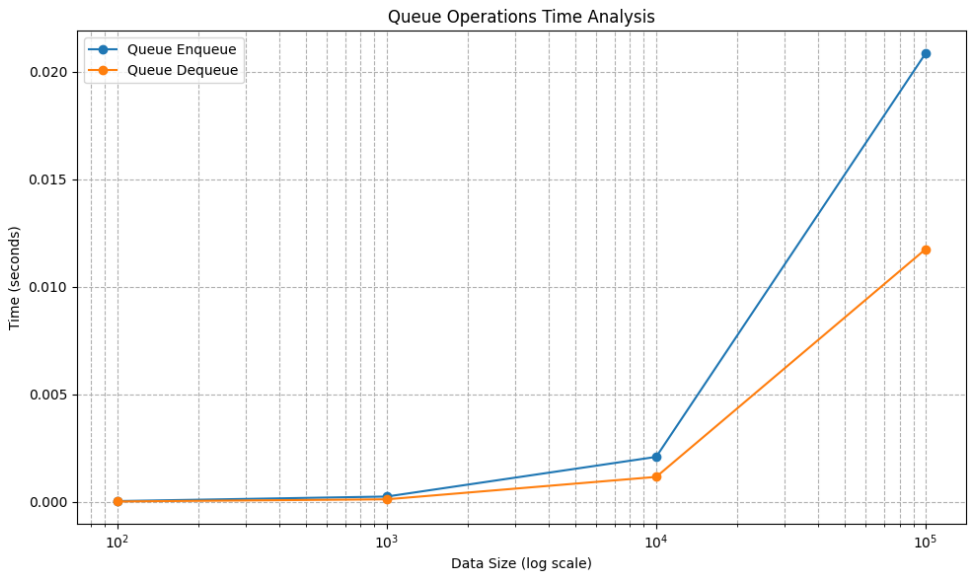

Figure 8: Performance of operations in queue data structure.

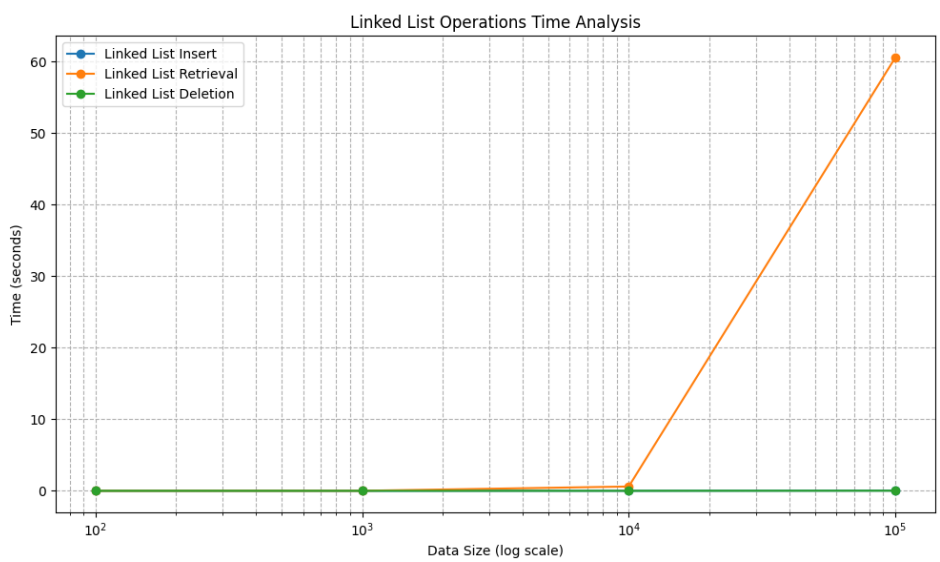

Figure 9: Performance of operations in linked list data structure.

References

Cormen, T., Leiserson, C., & Rivest, R. (2022). *Introduction to Algorithms* . Cambridge: The MIT Press.